

[정렬]

1. 선택 정렬 (Selection Sort)

- 개념: 정렬되지 않은 부분에서 가장 작거나(오름차순) 가장 큰(내림차순) 값을 찾아 정렬된 부분의 올바른 위치로 교환해나가는 방식입니다. 마치 빈 그릇에 가장 작은 돌멩이를 하나씩 골라 담는 것과 같습니다.
- 동작 방식:
 1. 정렬되지 않은 리스트에서 최솟값을 찾습니다.
 2. 찾은 최솟값을 정렬되지 않은 리스트의 첫 번째 요소(현재 정렬될 위치)와 교환합니다.
 3. 이 과정을 리스트 전체가 정렬될 때까지 반복합니다.
- 시간 복잡도: $O(N^2)$ (비교 횟수 및 이동 횟수 모두)
- 특징: 구현이 간단하지만, 데이터 양이 늘어날수록 효율이 떨어집니다. 불안정 정렬입니다.

2. 삽입 정렬 (Insertion Sort)

- 개념: 이미 정렬된 부분에 새로운 요소를 적절한 위치에 '삽입'하는 과정을 반복합니다. 마치 카드 게임에서 손에 든 카드들을 정렬하면서 새로운 카드를 받으면 제자리에 끼워 넣는 것과 같습니다.
- 동작 방식:
 1. 두 번째 요소부터 시작하여, 현재 요소를 그 앞의 정렬된 부분에 있는 요소들과 비교하며 적절한 위치를 찾습니다.
 2. 적절한 위치를 찾으면 해당 위치에 삽입하기 위해 그 뒤의 요소들을 한 칸씩 뒤로 이동시킵니다.
 3. 이 과정을 모든 요소에 대해 반복합니다.
- 시간 복잡도:
 - 최선: $O(N)$ (이미 정렬되어 있을 때)
 - 최악: $O(N^2)$ (역순으로 정렬되어 있을 때)
- 특징: 안정적 정렬입니다. 이미 정렬되어 있는 경우 매우 효율적이지만, 레코드 이동이 많아 레코드 크기가 크면 불리합니다.

3. 버블 정렬 (Bubble Sort)

- 개념: 인접한 두 요소를 비교하여 순서가 맞지 않으면 서로 교환하는 과정을 반복합니다. 마치 거품(bubble)이 물 위로 떠오르듯, 가장 큰(또는 작은) 값이 한 번의 스캔으로 맨 끝으로 이동하는 방식입니다.
- 동작 방식:
 1. 첫 번째 요소부터 인접한 두 요소를 비교합니다.
 2. 순서가 맞지 않으면 교환합니다.
 3. 이 과정을 리스트의 끝까지 반복하면, 가장 큰(또는 작은) 값이 맨 끝으로

이동합니다.

4. 정렬되지 않은 나머지 요소들에 대해 이 과정을 반복합니다.

- 시간 복잡도: $O(N^2)$ (최선, 평균, 최악 모두 동일)
- 특징: 레코드 이동이 과다하여 비효율적입니다. 안정적 정렬입니다.

4. 셸 정렬 (Shell Sort)

- 개념: 삽입 정렬의 단점(요소들의 이동이 인접한 위치로만 제한된다는 점)을 보완하기 위해 고안되었습니다. 불연속적인 부분 리스트를 정렬하여 원거리에 있는 요소들도 빠르게 이동시킬 수 있습니다.
- 핵심 원리:
 - 갭(Gap) 또는 간격: 전체 리스트를 일정한 간격으로 나누어 여러 개의 부분 리스트를 만듭니다.
 - 삽입 정렬 활용: 이 불연속적인 부분 리스트에 대해 삽입 정렬을 적용합니다.
 - 점진적인 간격 감소: 갭은 점차적으로 줄어들어 마지막에는 1이 됩니다. 갭이 1이 되면 일반적인 삽입 정렬과 동일하게 동작하지만, 이전에 부분적으로 정렬된 상태 덕분에 효율이 극대화됩니다.
- 시간 복잡도:
 - 평균: $O(N^{1.5})$
 - 최악: $O(N^2)$
- 특징: 삽입 정렬의 원거리 자료 이동 제한을 극복합니다. 불안정 정렬입니다.

5. 병합 정렬 (Merge Sort)

- 개념: '분할 정복(Divide and Conquer)' 기법을 사용하는 대표적인 정렬 알고리즘입니다. 큰 문제를 작은 문제로 나누어 해결하고, 그 결과를 합치는 방식입니다.
- 동작 방식:
 1. 분할 (Divide): 정렬할 리스트를 더 이상 나눌 수 없을 때(요소가 1개가 될 때)까지 절반으로 계속 나눕니다.
 2. 정복 (Conquer): 각 부분 리스트를 재귀적으로 정렬합니다. (요소가 1개인 리스트는 이미 정렬된 것으로 간주).
 3. 결합 (Combine) 또는 병합 (Merge): 정렬된 두 개의 부분 리스트를 합쳐 하나의 정렬된 리스트로 만듭니다. 실제로 정렬이 이루어지는 시점은 이 병합 단계입니다.
- 시간 복잡도: 최선, 평균, 최악 모두 $O(N \log N)$ (항상 일정)
- 특징: 안정적 정렬입니다. 추가적인 임시 배열 공간이 필요하므로 메모리 사용량이 많습니다.

6. 퀵 정렬 (Quick Sort)

- 개념: '평균적으로 가장 빠른 정렬 방법'으로 알려져 있으며, 병합 정렬과 마찬가지로 분할 정복 기법을 사용합니다.

- 핵심 원리:
 - 피벗 (**Pivot**): 리스트를 두 개의 부분 리스트로 분할하기 위한 기준 값입니다. 강의에서는 '가장 왼쪽에 있는 숫자'를 피벗으로 설정했습니다.
 - 분할 (**Partition**): 피벗을 기준으로 리스트를 두 부분으로 나눕니다. 피벗보다 작은 값들은 피벗의 왼쪽에, 큰 값들은 오른쪽에 위치시킵니다.
 - 재귀 호출: 분할된 각 부분 리스트에 대해 재귀적으로 퀵 정렬을 호출하여 정렬을 완료합니다.
- 시간 복잡도:
 - 평균: $O(N\log N)$ (매우 효율적)
 - 최악: **$O(N^2)$** (피벗 선택이 매우 비효율적일 때 발생, 예: 이미 정렬된 리스트에서 항상 첫 번째 요소를 피벗으로 선택하는 경우)
- 특징: 별도의 임시 배열이 필요하지 않아 메모리 효율적입니다. 불안정 정렬입니다.

7. 힙 정렬 (Heap Sort)

- 개념: 힙 자료구조를 활용한 정렬 알고리즘입니다.
- 핵심 원리:
 - 힙 (**Heap**): 완전 이진 트리 형태의 자료구조로, 최대 힙(부모 노드가 자식 노드보다 항상 큼) 또는 최소 힙(부모 노드가 자식 노드보다 항상 작음)으로 구성됩니다.
 - 정렬 과정: 전체 요소를 힙으로 구성한 후, 힙에서 한 번에 하나의 요소(최대 힙의 경우 루트, 즉 가장 큰 값)를 삭제하여 정렬된 배열에 저장하는 방식으로 정렬합니다.
- 시간 복잡도: $O(N\log N)$ (힙 생성 및 N 번의 삭제 연산)
- 특징: 전체 자료를 모두 정렬할 필요 없이, 가장 큰 값 몇 개 또는 가장 작은 값 몇 개만 필요할 때 유용합니다. 불안정 정렬입니다.

8. 기수 정렬 (Radix Sort)

- 개념: 레코드를 비교하지 않고, 자릿수(또는 문자 위치)를 기준으로 버킷(임시 공간)을 활용하는 정렬 방법입니다.
- 핵심 원리:
 - 버킷 (**Bucket**): 임시 저장 공간으로, 각 자릿수(0~9) 또는 문자(A~Z)에 해당하는 버킷을 만듭니다. (주로 큐로 구현)
 - 낮은 자릿수부터 정렬: 가장 낮은 자릿수(예: 1의 자리)부터 시작하여 각 숫자를 해당 버킷에 넣고 순서대로 다시 꺼냅니다.
 - 반복: 다음 자릿수에 대해 이 과정을 반복합니다.
- 시간 복잡도: $O(DN)$ (D 는 키의 자릿수, N 은 레코드 수). 키의 자릿수 D 가 작을수록 매우 빠릅니다.

- 특징: 비교 연산을 사용하지 않습니다. 안정적 정렬입니다. 정렬할 레코드 타입이 한정적입니다 (숫자 또는 문자로 구성되고 동일한 길이를 가져야 함). 실수, 한글, 한자 등에는 적용하기 어렵습니다.